

Physics-informed neural networks combined with polynomial interpolation to solve nonlinear partial differential equations

Siping Tang^a, Xinlong Feng^{a,*}, Wei Wu^a, Hui Xu^{b,*}

^a College of Mathematics and System Sciences, Xinjiang University, Urumqi 830046, PR China

^b School of Aeronautics and Astronautics, Shanghai Jiao Tong University, Shanghai 200240, PR China

ARTICLE INFO

Keywords:

Machine learning
Physics-informed neural networks
Partial differential equations
Polynomial interpolation

ABSTRACT

In this paper, we utilise the physics-informed neural networks (PINN) combined with interpolation polynomials to solve nonlinear partial differential equations and for simplicity, the resulted neural network is termed as *polynomial interpolation physics-informed neural networks* (PI-PINN). Classically, the neural network is expressed as a power series by optimization of the coefficients to get an approximate solution of the partial differential equations (PDEs). Due to well-defined approximate properties of orthogonal polynomials, orthogonal polynomials are used to construct the neural network. Compared with PINN, PI-PINN clearly has a simple structure and is easy to be understood. We carry out some numerical experiments, including parabolic partial differential equations, hyperbolic partial differential equations and an application in fluid mechanics. By these investigations, it is further demonstrated that the PI-PINN structure is effective in solving nonlinear partial differential equations. Further, investigations on reverse problems are taken and accurate results are obtained.

1. Introduction

Machine learning has been significantly developed during last decade, which is prompted by the development of computer technology, data science and mathematical theory in neural network [1–4]. Various methods developed in neural network have demonstrated that using neural network is able to speed up both fundamental and applied researches in science and engineering. With the development of deep learning, solving partial differential equations (PDEs) by machine learning has been growing rapidly. As early as 1989, K. Hornik *et al.* established the universal approximation theorem for neural network [5], which theoretically explained that neural network could approach any continuous function. At the same time, Lagaris *et al.* solved the problem of the initial boundary value through a neural network [6]. However, due to the limitation of computer resources, the related researches did not draw too much attention. Recently, Karniadakis' research team introduced the concept of the Physics-Informed Neural Networks (PINN) [7,8], in which the physical laws described by nonlinear partial differential equations are embed in loss function to successfully solve forward and inverse problems. Subsequently, a series of novel learning algorithms combined with PINN were proposed. Typically, Bayesian Physics-Informed Neural Networks (B-PINN) was

designed to solve forward and inverse problems of partial differential equations where noise data is present [9,10]. Kovacs *et al.* proposed conditional Physics-Informed Neural Networks [11] for eigenvalue problems of partial differential equation and the corresponding algorithms combined with domain decomposition were designed [12–16]. In applications of PINN, Mao *et al.* applied PINN to approximate the Euler equations for high-speed aerodynamic flows [17]. Wang *et al.* constructed a new architecture using spatio-temporal and multi-scale random Fourier features to solve multi-scale problems [18]. Jin *et al.* solved the problem of laminar flows and channel turbulence by establishing NSF-nets in the form of velocity-pressure and velocity-vorticity [19]. Lou *et al.* combined the Boltzmann-BGK formulation with PINN to solve forward and inverse flow [20], and the application in some other subject areas [21–26].

As we know, when neural network is used to solve complex problems, it suffers from slow convergence. To alleviate this problem, the adaptive activation function [27–29] and the adaptive weight [30–32] have been proposed. The adaptive activation function is used to accelerate convergence and improve accuracy by adding additional superparameters to the activation function; The adaptive weight utilises gradient statistics to balance interaction between different components in loss function by adding weight during training.

* Corresponding authors.

E-mail addresses: 2571072734@qq.com (S. Tang), fxlmath@xju.edu.cn (X. Feng), wuwei837037@163.com (W. Wu), dr.hxu@sjtu.edu.cn (H. Xu).

<https://doi.org/10.1016/j.camwa.2022.12.008>

Received 11 March 2022; Received in revised form 20 October 2022; Accepted 8 December 2022

Available online 19 December 2022

0898-1221/© 2022 Elsevier Ltd. All rights reserved.

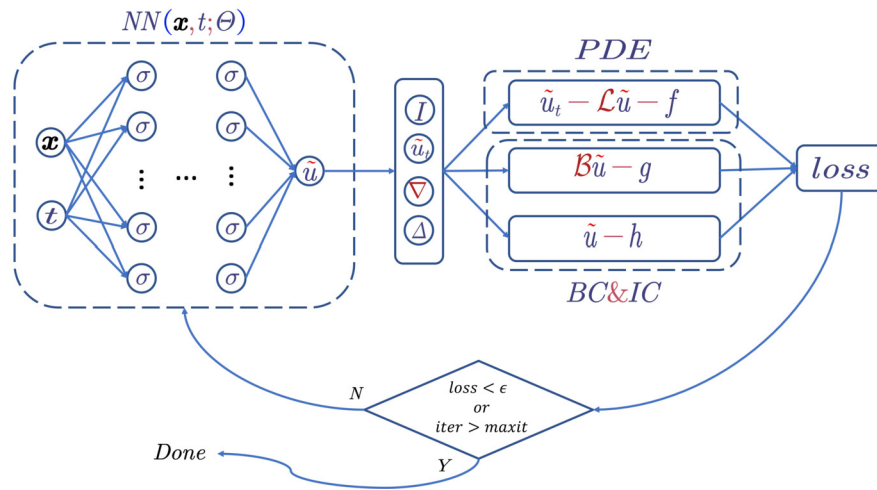


Fig. 1. Sketch of PINN for Eq. (2.2).

In the resolution of partial differential equations with neural network, it is inevitable to adjust certain hyperparameters in neural networks (depth, width, and so on) in order to achieve a good effect, and this process is long. In most cases, selecting the right network structure according to the researcher's experience is a trial and error approach. Therefore, we hope to construct a simple network structure to reduce the adjustment of network parameters. Considering Fourier neural networks with only one hidden layer [33,34], based on the similar idea, combining neural network and interpolation polynomial, we here construct *Polynomial Interpolation Physics-Informed Neural Networks* (PI-PINN). In terms of function approximation, an arbitrary function can be expanded into a polynomial series. However, it is not easy to calculate expansion coefficients with high efficiency. Therefore, as investigated by Yang et al. [35] and Mall et al. [36] based on orthogonal polynomials, we also use neural network to learn each coefficient in the expansion and in turn, we try to obtain approximate solutions of nonlinear partial differential equations.

The structure of this paper is organised as follows. In §2, we give a brief introduction of Physics-Informed Neural Networks. In §3, we introduce in this paper, the Polynomial Interpolation Physics-Informed Neural Networks. In §4, we give some examples including forward problem and inverse problem to validate the proposal neural network and demonstrate the network structure is effective in solving nonlinear partial differential equations. Finally, in §5, we summarize the work.

2. Physics-informed neural network

Mathematically, we can consider a deep neural network as a composite function of the entry variable $\mathbf{x}$, with a very strong approximation capability. For a neural network with the depth of H , it has an input layer, $(H - 1)$ hidden layers, and an output layer. We assume that there are N_k neurons in the k -th hidden layer, and the corresponding weight matrix and bias vector are $W^k \in R^{N_k \times N_{k-1}}$ and $b^k \in R^{N_k}$, respectively. Given a nonlinear activation function σ , the neural network is described recursively as follows [27]:

$$\tilde{u}_\Theta(\mathbf{x}) = W^H(\sigma(W^{H-1}(\dots \sigma(W^2(\sigma(W^1\mathbf{x} + b^1)) + b^2) \dots) + b^{H-1})) + b^H, \quad (2.1)$$

where $\tilde{u} = \tilde{u}_\Theta(\mathbf{x})$ is the output of the neural network and the approximate solution of the partial differential equations, $\Theta = \{W^i, b^i\}_{i=1}^H$ are the trainable parameters in the neural network. See also Fig. 1 for visualization of a neural network.

A Physics-Informed Neural Networks (PINN) in which the partial differential equation and its initial boundary conditions are embedded

into the loss function of the neural network by the automatic differentiation technology [37]. A partial differential equation of the following form is taken into account [38]:

$$\begin{cases} u_t - \mathcal{L}u = f, (\mathbf{x}, t) \in \Omega \times [0, T], \\ Bu = g, (\mathbf{x}, t) \in \partial\Omega \times [0, T], \\ u = h, (\mathbf{x}, t) \in \Omega \times \{0\}, \end{cases} \quad (2.2)$$

where $\Omega \subset R^d$ ($d = 1, 2, 3$) is the d -dimensional physical domain, $\mathcal{L}$ representation linear or nonlinear differential operator and Bu is the boundary condition, including but not limited to Dirichlet boundary condition, Robin boundary condition.

In the training set, we generally use the mean square error (MSE) to define the loss function of Eq. (2.2), it has the following form [39]:

$$\begin{aligned} loss = & \frac{1}{N_f} \sum_{i=1}^{N_f} |\tilde{u}_t(\mathbf{x}^i, t^i) - \mathcal{L}\tilde{u}(\mathbf{x}^i, t^i) - f(\mathbf{x}^i, t^i)|^2 \\ & + \frac{1}{N_b} \sum_{j=1}^{N_b} |B\tilde{u}(\mathbf{x}^j, t^j) - g(\mathbf{x}^j, t^j)|^2, \\ & + \frac{1}{N_u} \sum_{k=1}^{N_u} |\tilde{u}(\mathbf{x}^k, 0) - h(\mathbf{x}^k)|^2 \end{aligned} \quad (2.3)$$

where $\{\mathbf{x}^i, t^i\}_{i=1}^{N_f}$ are the residual training points in the solution region of time and space, $\{\mathbf{x}^j, t^j\}_{j=1}^{N_b}$ are the points on the boundary of the solution region, and $\{\mathbf{x}^k, t^k\}_{k=1}^{N_u}$ are the initial training points. The first term in the formula (2.3) represent the physical model-driven part of the loss function, while the second and third terms represents the data-driven part of the loss function. PINN is an approach which combination of model-driven and data-driven. Fig. 1 shows a sketch of a PINN algorithm [39] for Eq. (2.2).

For the simple initial condition and boundary condition, we can use the strategy of the forced constraints [6,40] to make the approximate solution hold precisely at the boundary and the initial time. For example, supposing a partial differential equation is defined on the interval $[0, 1]$ with Dirichlet boundary condition: $u(0) = a$, $u(1) = b$, we can construct its approximate solution by enforcing a strategy that constrains the boundary conditions: $u^* = x(x - 1)\tilde{u} + (b - a)x + a$, or $u^* = \sin(\pi x)\tilde{u} + (b - a)x + a$, etc. So, we can construct various approximate solutions by using the strategy of boundary constraints.

When PINN is used to solve partial differential equations, the purpose is to find the optimal parameters Θ for which the suitably defined

loss function is minimized. Gradient optimization algorithms (SGD [41], MBGD [42], Adam [43], etc.) or Newton-type optimization algorithms (BFGS, L-BFGS [44], etc.) can be applied to the training process and the parameters are updated to obtain a local optimal value of the loss function.

3. Polynomial interpolation physics-informed neural networks

3.1. Power series

In this part, the Physics-Informed Neural Networks combined with polynomial interpolation used in this paper is introduced, and we regard it as *Polynomial Interpolation Physics-Informed Neural Networks* (PI-PINN).

PINN feeds the input data into a neural network as a whole and the proposed method splits the input data into two independent parts because we need to perform operations separately,

$$(x, y) \longrightarrow (x), (y). \quad (3.1)$$

We're given a row vector T of length $(n+1)$, where all the elements are 1 and we use a fixed vector instead of the weight matrix in PINN. So we get two new vectors:

$$\begin{aligned} (x) &\longrightarrow (x, x, \dots, x) = A \in \mathbb{R}^{1 \times (n+1)}, \\ (y) &\longrightarrow (y, y, \dots, y) = B \in \mathbb{R}^{1 \times (n+1)}. \end{aligned} \quad (3.2)$$

The idea of the adaptive activation function can make T as a vector with all initial parameters 1 and the parameters can be updated in the neural network training process. These two vectors A and B will be used as inputs for the neurons of the hidden layer. We take different powers operations off each of these two vectors and we get two new vectors, and let's call the two vectors as X and Y ,

$$\begin{aligned} (x, x, \dots, x) &\longrightarrow (1, x, \dots, x^n) = X \in \mathbb{R}^{1 \times (n+1)}, \\ (y, y, \dots, y) &\longrightarrow (1, y, \dots, y^n) = Y \in \mathbb{R}^{1 \times (n+1)}. \end{aligned} \quad (3.3)$$

We use the power operation instead of the single activation function in PINN. In other words, we use different activation functions for each neuron in the x and y directions. Generally, applying the Kronecker product to vectors X and Y yields a $(1 \times (n+1)^2)$ vector (similar expression can be found in [35])

$$X \otimes Y = (Y, xY, \dots, x^n Y). \quad (3.4)$$

Multiplying each term of this vector by a weight coefficient, classically, we get the output of the neural network [35,36]

$$\tilde{u} = \sum_{i,j=0}^n w_{ij} x^i y^j. \quad (3.5)$$

In Fig. 2, it shows the framework of *Polynomial Interpolation Physics-Informed Neural Networks*. It can be seen that it is a shallow neural network with a single hidden layer, and the function space is generated by this neural network is a polynomial function space.

3.2. Orthogonal polynomials

In the above section, we use a set of linearly independent power functions as basis functions to construct a neural network. In fact, it approximates the exact solution of a partial differential equation by finding a polynomial in a polynomial function space. In the same way, we can use various polynomial functions to build our neural network. Orthogonal polynomials are very useful tools for approximation of partial differential equations. Therefore, it is natural to build a polynomial neural network based on orthogonal polynomials for solving partial differential equations. Yang et al. [35] solved elliptic partial differential equations by Legendre polynomials in the framework of extreme learning machine, and Mall et al. [36] used the Chebyshev polynomials of

the first kind to establish Chebyshev neural networks, and used to solve elliptic partial differential equations. For further validation in solving nonlinear partial differential equations, we here still use Legendre polynomial [35,45] and Chebyshev polynomials of the second kind [36,46] to build the similar neural networks.

Legendre polynomials are polynomials defined on the interval $[-1, 1]$, with weight function $\rho(x) \equiv 1$, and their expressions are given by [45]:

$$P_0(x) = 1, P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n, \quad n = 1, 2, \dots, \quad (3.6)$$

and we simply have the recursive formula for these polynomials:

$$\begin{aligned} P_0(x) &= 1, P_1(x) = x, \\ P_{n+1}(x) &= \frac{(2n+1)xP_n(x) - nP_{n-1}(x)}{n+1}, \quad n = 1, 2, \dots \end{aligned} \quad (3.7)$$

Therefore, the output of the neural network is formulated by:

$$\tilde{u}(x) = \sum_{n=0}^N w_n P_n(x). \quad (3.8)$$

For Chebyshev polynomial, we consider the Chebyshev polynomials of the second kind defined on the interval $[-1, 1]$, and the weight function $\rho(x) = \sqrt{1-x^2}$. It has the following form [46]:

$$U_n(x) = \frac{\sin[(n+1)\arccos(x)]}{\sqrt{1-x^2}}, \quad |x| \leq 1. \quad (3.9)$$

The recursive relation is given by:

$$\begin{aligned} U_0(x) &= 1, U_1(x) = 2x, \\ U_{n+1}(x) &= 2xU_n(x) - U_{n-1}(x), \quad n = 1, 2, \dots \end{aligned} \quad (3.10)$$

Similarly, the output of the neural network is defined by:

$$\tilde{u}(x) = \sum_{n=0}^N w_n U_n(x). \quad (3.11)$$

For the higher dimensional cases, we can use the Kronecker product to form its structure.

4. Numerical investigations

In this section, several examples are provided to verify the effectiveness of *Polynomial Interpolation Physics-Informed Neural Networks* in solving nonlinear partial differential equations. We introduce the L^2 error [47] to measure the difference between approximate solution $u^*(x, t)$ of neural network output and exact solution $u(x, t)$ of partial differential equations:

$$L^2 \text{ error} = \frac{\sqrt{\sum_{i=1}^N |u^*(x^i, t^i) - u(x^i, t^i)|^2}}{\sqrt{\sum_{i=1}^N |u(x^i, t^i)|^2}}, \quad (4.1)$$

where N represent the number of test points.

4.1. Forward problems

4.1.1. The Burgers equation

To verify the effectiveness of the neural networks, we first take the two-dimensional viscous Burgers equation as an example [48]:

$$\begin{aligned} u_t + f(u)_x + g(u)_y &= u_{xx} + u_{yy}, \quad (x, y, t) \in (-1, 1)^2 \times (0, 4), \\ u &= g, \quad (x, y, t) \in \partial(-1, 1)^2 \times (0, 4), \\ u &= h, \quad (x, y, t) \in (-1, 1)^2 \times \{0\}, \end{aligned} \quad (4.2)$$

where the inviscid fluxes in x and y directions are $f(u) = u^2/2$, and $g(u) = u^2/2$, respectively. For PI-PINN, the initial and boundary conditions are obtained from the exact solution $u(x, y, t) = 1/(1 +$

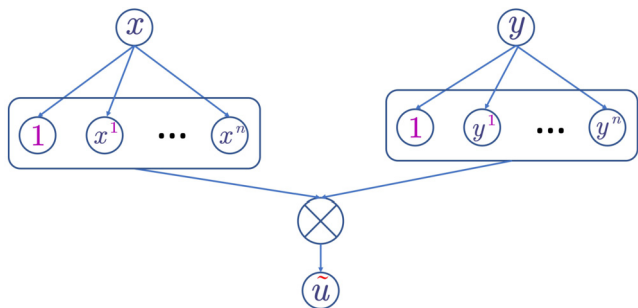


Fig. 2. Polynomial Interpolation Physics-Informed Neural Networks framework.

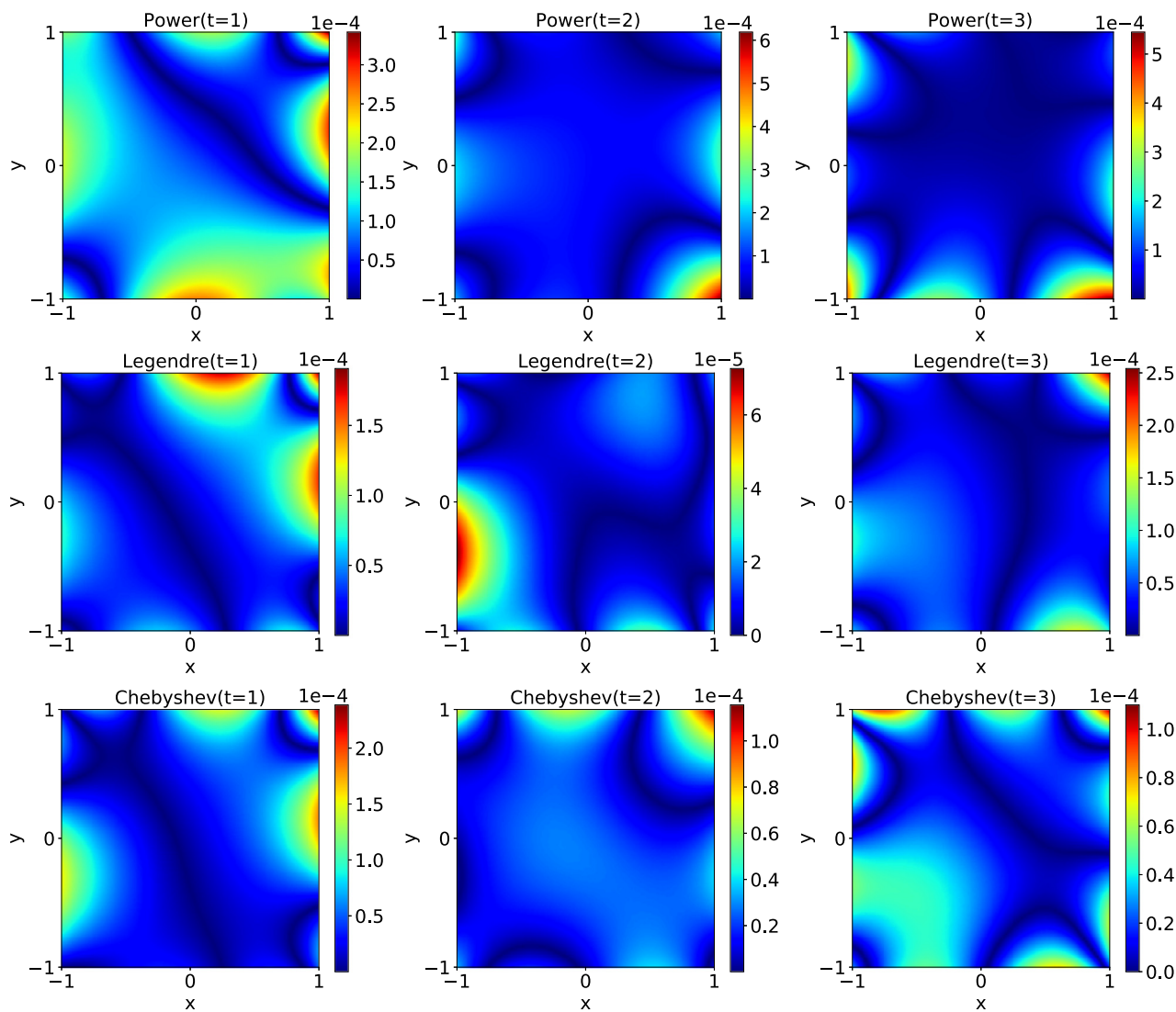


Fig. 3. Burgers equation: error distributions in three-time snapshots.

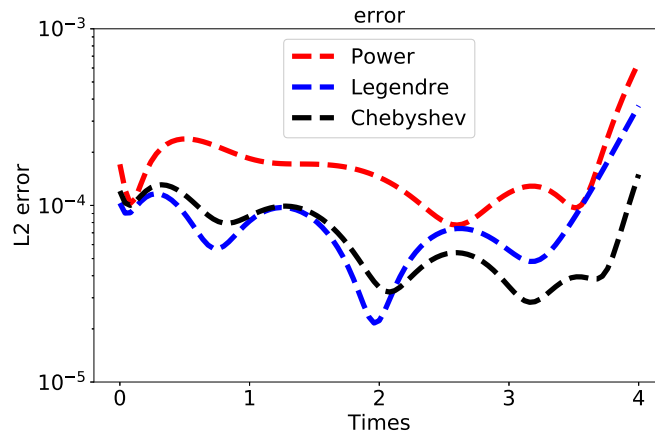


Fig. 4. Burgers equation: the curves of L^2 error with times.

$\exp((x+y-t)/2)$). Here, the goal is to infer the whole spatial-temporal solution u from the above Eq. (4.2). We select to represent the u latent function together using 5 neurons in each temporal and spatial directions. In general, the neural network should have enough approximation capacity to consider the predicted complexity of u . This training dataset includes a total of 200 data points at the boundary and at the initial time. $N_f = 500$ randomly sampled collocation points are used to apply to Eq. (4.2) in the solution area. All randomly sampled point locations were generated using a space filling Latin Hypercube Sampling strategy [49]. Fig. 3 shows the error distributions of three-time snapshots at $t = 1.0$, $t = 2.0$, and $t = 3.0$, using Power function, Legendre polynomial, and Chebyshev polynomial, respectively. In Fig. 4, we illustrate the L^2 error curves of Eq. (4.2) on the test set. Similarly, the calculation time of power series, Legendre polynomials and Chebyshev polynomials is 79.1960s, 74.7176s, and 174.7666s, respectively. From the results, it is seen that the PI-PINN can produce the solution with a good precision.

4.1.2. The Sine-Gordon equation

We take the two-dimensional unsteady Sine-Gordon equation [50] as an example to further validate the effectiveness of PI-PINN in solving nonlinear hyperbolic partial differential equations. It is given as follows:

$$\begin{aligned} u_{tt} - (u_{xx} + u_{yy}) - \sin(u) &= f, (x, y; t) \in (0, 1)^2 \times (0, 2), \\ u &= g, (x, y; t) \in \partial(0, 1)^2 \times (0, 2), \\ u &= h_1, (x, y; t) \in (0, 1)^2 \times \{0\}, \\ u_t &= h_2, (x, y; t) \in (0, 1)^2 \times \{0\}. \end{aligned} \quad (4.3)$$

The exact solution of this equation is assumed to have the form $u = \exp(-t) \sin(\pi x) \sin(\pi y)$, and we can get the right-hand terms correspondingly. In this example, we use the boundary constraint strategy to construct an approximate solution. 6 neurons are set in the x , y , and t directions. The training dataset is made up of $N_u = 100$ initial data points which are randomly sampled from the exact solution at time $t = 0$, and $N_f = 600$ randomly sampled collocation points, and the approximate solution is constructed as $u^* = xy(x-1)(y-1)\tilde{u}$. The error contours of three-time snapshots of Eq. (4.3) are shown in Fig. 5 for $t = 0.5$, $t = 1.0$ and $t = 1.5$. In Fig. 6, it shows the L^2 error. Similarly, the calculation time of power series, Legendre polynomials and Chebyshev polynomials is 58.1761s, 95.7654s, and 257.0125s. It is found that there exists a good approximation of the exact solutions.

4.1.3. The Allen-Cahn equation

The Allen-Cahn equation is a second-order nonlinear parabolic partial differential equation, used to describe the phase separation process in binary alloys and the anti-phase boundary motion in crystals [51].

It has been widely used in a variety of models, such as the interface dynamics model in materials science [52], crystal growth [53], phase-field model of tumor growth [54], etc. Consider the three-dimensional Allen-Cahn equation as an example:

$$\begin{aligned} u_t - (u_{xx} + u_{yy} + u_{zz}) + u^3 - u &= f, (x, y, z; t) \in (0, 1)^3 \times (0, 2), \\ u &= g, (x, y, z; t) \in \partial(0, 1)^3 \times (0, 2), \\ u &= h, (x, y, z; t) \in (0, 1)^3 \times \{0\}. \end{aligned} \quad (4.4)$$

We take $u = \exp(-t) \sin(\pi x) \sin(\pi y) \sin(\pi z)$ as the exact solution, and both the source term and the initial boundary condition are given by the exact solution. We can use the boundary constraint strategy to construct an approximate solution of $u^* = xyz(x-1)(y-1)(z-1)\tilde{u}$. For PI-PINN, we employ 200 initial training points, 1000 residual points, and 5 neurons are used respectively in temporal and spatial directions. In Fig. 7, it shows the error for $t = 0.5$, $t = 1.0$, and $t = 1.5$, and $z = 0.5$. Similarly, the L^2 error are given in Fig. 8. The calculation time of the power series, Legendre polynomials and Chebyshev polynomials can be obtained as 63.2363s, 111.3321s, and 332.5545s. It is clearly observed that solutions with a good precision are obtained for this equation.

4.1.4. The Kovasznay flow

We further use PI-PINN to solve two-dimensional steady-state incompressible Kovasznay flow [55] in a rectangular domain with a length $-0.5 \leq x \leq 1$ and height $-0.5 \leq y \leq 1.5$. The governing equations are given by:

$$\begin{aligned} u_x + v_y &= 0, \\ uu_x + vv_x &= -p_x + \nu(u_{xx} + u_{yy}), \\ uv_x + vv_y &= -p_y + \nu(v_{xx} + v_{yy}), \end{aligned} \quad (4.5)$$

where ν denotes kinematic viscosity of the fluid. The exact solutions considered are given by $u = 1 - \exp(\xi x) \cos(2\pi y)$, $v = \frac{\xi}{2\pi} \exp(\xi x) \sin(2\pi y)$, and $p = \frac{1}{2} (1 - \exp(2\xi x))$, here $\xi = \frac{1}{2\nu} - \sqrt{\frac{1}{4\nu^2} + 4\pi^2}$, $\nu = \frac{1}{Re}$. In Table 1, it shows the L^2 errors and a comparison is given between the results of PINN and Power function. With PI-PINN, the accuracy of pressure p is improved by one order of magnitude, although the accuracy of the first velocity component u is not significantly improved. In Fig. 9, it shows the predicted solutions, the exact solutions, and the error contours by Power function. For $Re = 40$, we obtain the training time of the power series, Legendre polynomials and Chebyshev polynomials, which are 3880.9975s, 1814.9150s and 3465.6869s, respectively. In Table 2 and Table 3, the corresponding L^2 errors of Legendre polynomial and Chebyshev polynomial are shown for different Reynolds numbers.

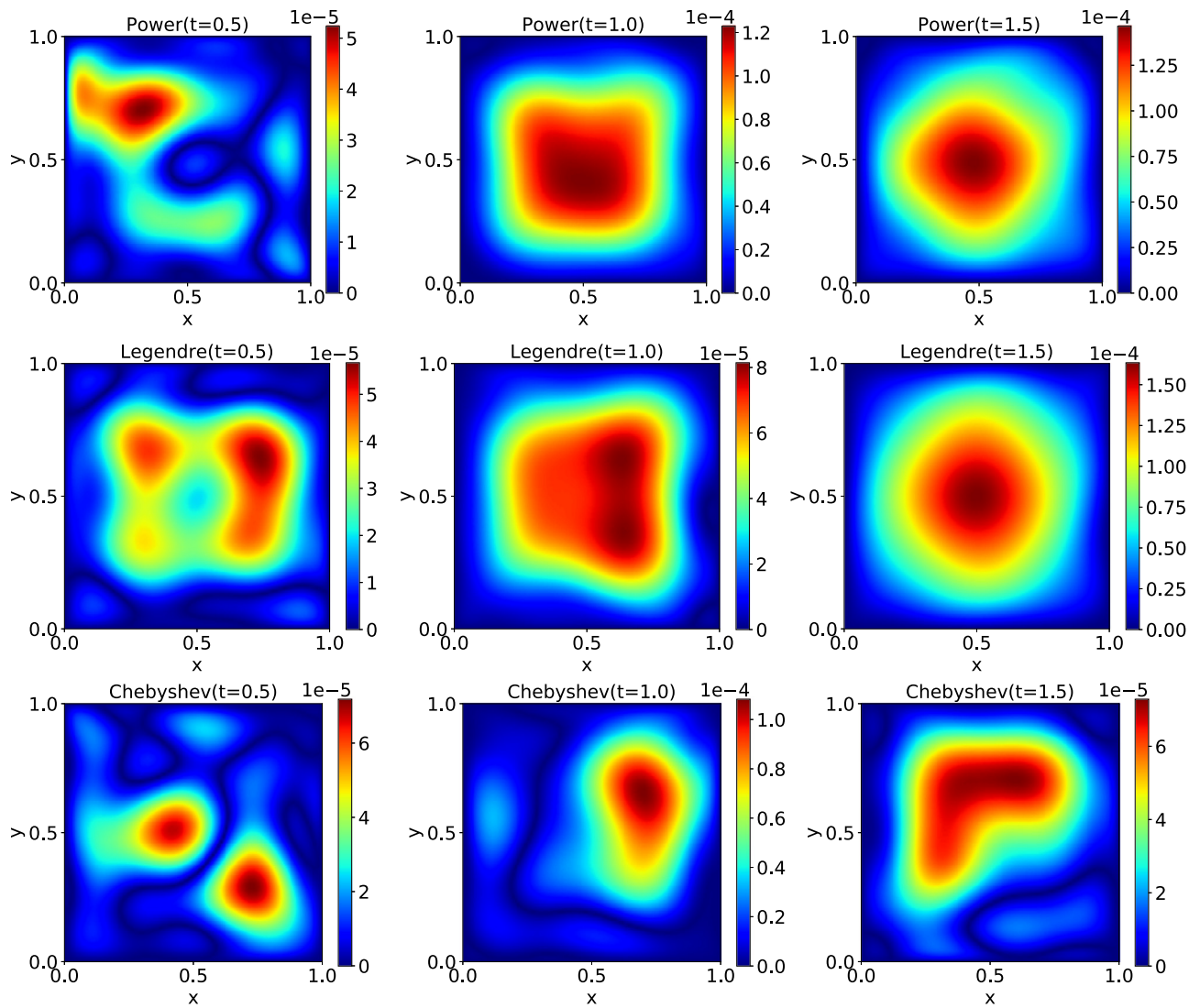


Fig. 5. Error contours for three snapshots at different time.

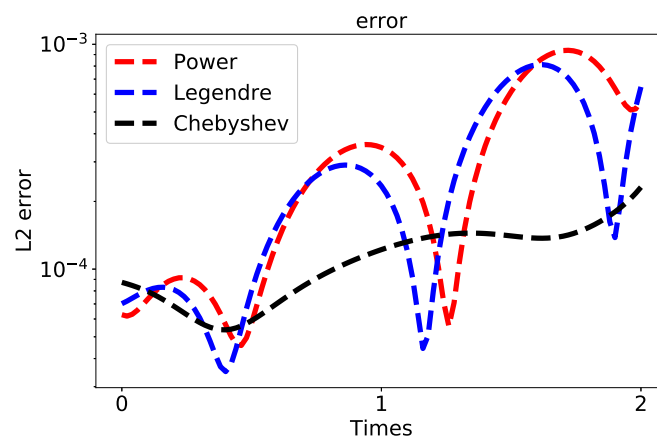


Fig. 6. The L^2 error with time evolution.

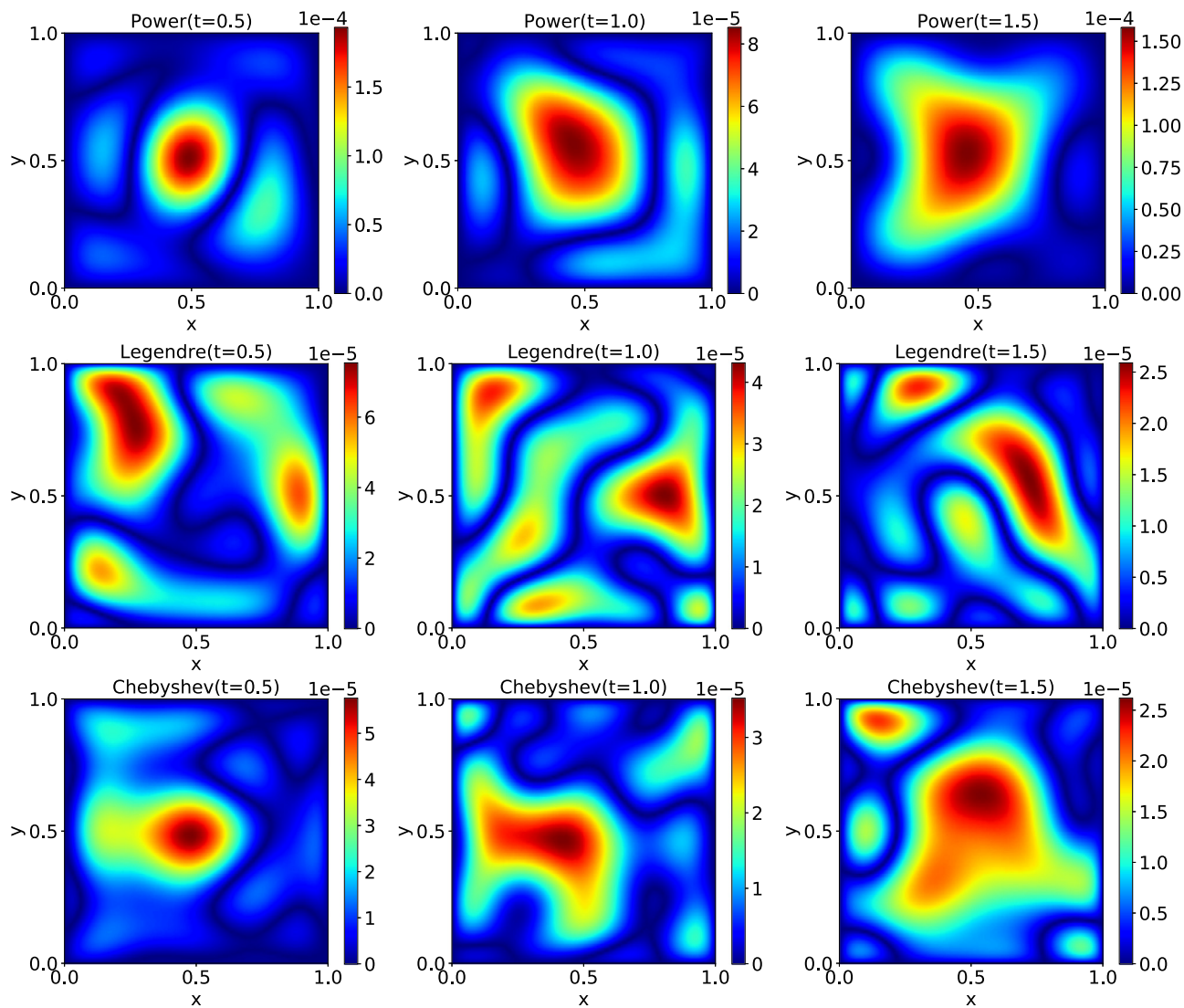


Fig. 7. Allen-Cahn equation: error distributions in three-time snapshots, and $z = 0.5$.

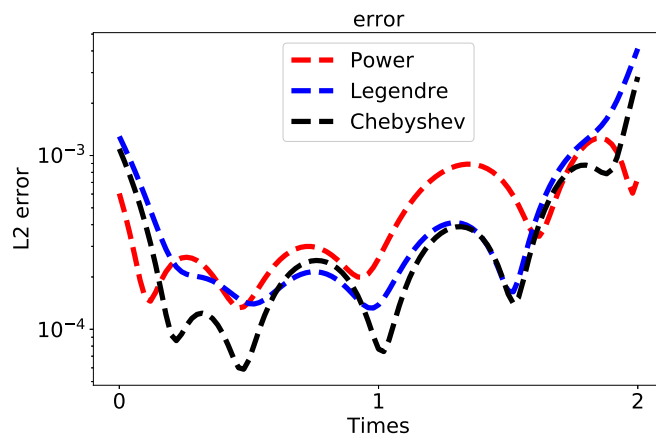


Fig. 8. Allen-Cahn equation: the curves of L^2 error with times.

Table 1
Comparison of L^2 errors between PINN and Power function.

Re = 40	PINN			Power		
	u	v	p	u	v	p
L^2 error	2.1895e-04	1.4257e-03	7.5644e+00	1.5078e-03	6.2470e-03	3.3868e-01

Table 2
Legendre polynomial is used to solve Eq. (4.5) and L^2 errors at different Reynolds numbers.

Re	40	60	100	200	500	1000
error u	2.4118e-05	6.2941e-05	4.5766e-05	4.0008e-05	7.4265e-05	3.7399e-05
error v	1.7485e-04	7.6281e-04	5.7876e-04	2.7473e-04	1.3197e-03	2.5722e-03
error p	2.7304e-01	7.0161e-01	2.1210e-01	2.3785e-01	5.0944e-01	2.0156e+00

Table 3
Chebyshev polynomial is used to solve Eq. (4.5) and L^2 errors at different Reynolds numbers.

Re	40	60	100	200	500	1000
error u	3.1776e-05	4.8809e-05	3.3601e-05	3.4718e-05	3.0747e-05	9.2433e-05
error v	1.2712e-04	1.8521e-04	4.1648e-04	6.0533e-04	1.4060e-03	2.3898e-03
error p	3.6662e-01	3.7059e-01	5.5367e-01	2.6821e-01	3.8533e+00	2.4459e+00

4.1.5. Flow in a Lid-driven cavity

In last example for forward problems, we consider a classical benchmark problem in computational fluid dynamics, the steady-state flow in a two-dimensional Lid-driven cavity flow [56]. The governing equation is given by Eq. (4.5), and the boundary conditions are given as:

$$\begin{aligned} u(x, 0) = 0, u(x, 1) = 1, u(0, y) = 0, u(1, y) = 0, \\ v(x, 0) = 0, v(x, 1) = 0, v(0, y) = 0, v(1, y) = 0. \end{aligned} \quad (4.6)$$

In this example, we have chosen the Reynolds number of $Re = 100$. Our goal here is to train a neural network to predict the latent velocity and pressure fields. From the boundary conditions (4.6), we can construct the approximate solution as: $u^* = xy(x-1)\tilde{u}$ and $v^* = xy(x-1)(y-1)\tilde{v}$. For neural network, we give 15 neurons in each spatial direction and construct a set with 5,000 residual training points and 1,000 upper boundary training points to simulate the fluid flow problem described in Eq. (4.5). We show the simulation results in Fig. 10. In addition, we compare the centerline velocity in Fig. 11.

Above, we use three kinds of neural networks constructed by polynomial series to simulate the Lid-driven cavity flow when $Re = 100$. Next, we use only power series to construct neural networks and simulate the Lid-driven cavity flow with Reynolds numbers of 300, 500, 600, and 1,000, respectively. Here, the approximate solutions are constructed as $u^* = y\sin(\pi x)\tilde{u}$ and $v^* = \sin(\pi x)\sin(\pi y)\tilde{v}$. For $Re = 300$, there are 30 neurons in each spatial direction, for Reynolds number is 500 and 600, there are 40 neurons in each spatial direction, and for $Re = 1,000$ is 60. We use 10,000 residual training point, and 1,000 training points on the upper boundary to train the neural network. The training results are shown in Fig. 12 and Fig. 13.

4.2. The inverse problem

4.2.1. The heat equation

The heat equation [57] is a parabolic partial differential equation, which is one of the basic equations in mathematical physics. The heat equation and its variations are important in applied science and mathematics. Let us take the one-dimensional nonlinear heat equation of the following form:

$$u_t - au_{xx} - b\cos(x)u^2 = f, \quad (x, t) \in (-\pi, \pi) \times (0, 1), \quad (4.7)$$

where the source term f can be obtained by the exact solution $u = \exp(-2t)\sin(x)$. To demonstrate how effective the approach is, we create a training dataset by randomly generated $N = 500$ points on the whole

space-time domain from the exact solution with $a = 2$ and $b = 4$, and in time and space directions, we employ 10 neurons. In Fig. 14, it shows the exact solution and the training sample points. It is seen in Table 4 that, for all three cases respectively with clear data, 1% noise data and 5% noise data, the predicted results by the PI-PINN are quite accurate. In Fig. 15, it shows the parameters a and b with the iterations. It is observed that results with good precision are obtained.

4.2.2. The Kolmogorov-Petrovsky-Piskunov equation

We further consider the Kolmogorov-Petrovsky-Piskunov equation (the KPP equation) given by [58]

$$u_t - Du_{xx} - au - bu^m = 0, \quad (x, t) \in (-\pi, \pi) \times (0, 1), \quad (4.8)$$

where a, b, D , and m are constants, and $m \neq 1$. The KPP equation has an exact solution as follows:

$$u(x, t) = [\beta + \exp(\lambda t + \frac{vx}{\sqrt{D}})]^{\frac{2}{1-m}},$$

where the parameters are given by

$$\begin{aligned} \beta &= \sqrt{-\frac{b}{a}}, \\ v &= \sqrt{\frac{a(1-m)^2}{2(m+1)}}, \\ \lambda &= \frac{a(1-m)(m+3)}{2(m+1)}. \end{aligned}$$

We set $D = 1$, $m = 2$ and parameters a and b as unknown parameters. We use 10 neurons in temporal and spatial directions to demonstrate the ability of the proposed method by learning from sparse and rare training data. We select the number of train points $N = 500$ to ‘find’ unknown parameters a and b . The results are show in Table 5. In Fig. 16, it shows the exact solution and the training points used. In Fig. 17, it shows the variation of obtained unknown parameters a and b by using neural network training. From the approximation errors, it shows that by the proposed method, a high precision is achieved.

4.2.3. The Diffusion-reaction equation

We finally consider a 3D steady-state Diffusion-reaction [9] equation given by

$$\lambda(u_{xx} + u_{yy} + u_{zz}) - ku^2 = f, \quad (4.9)$$

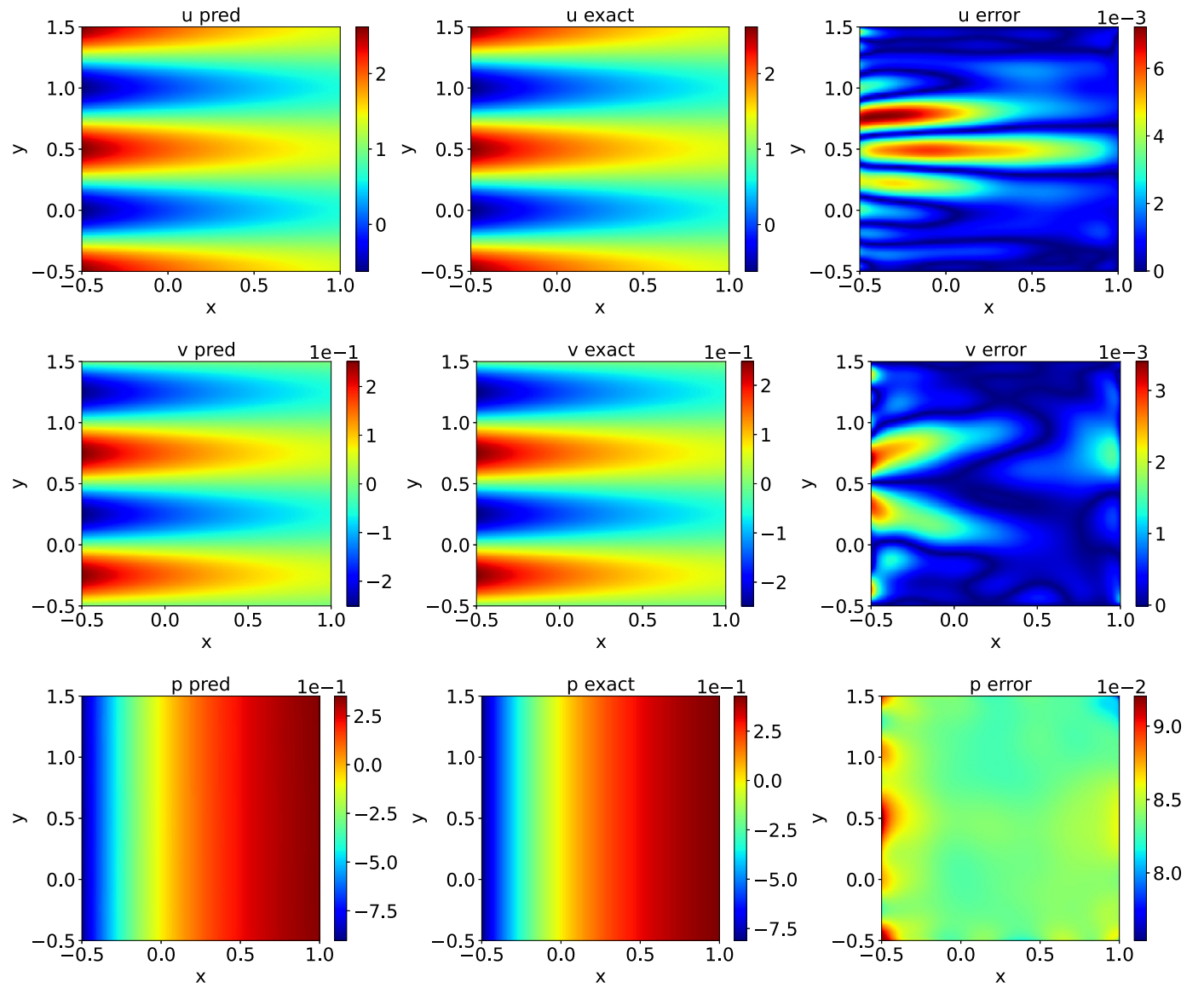
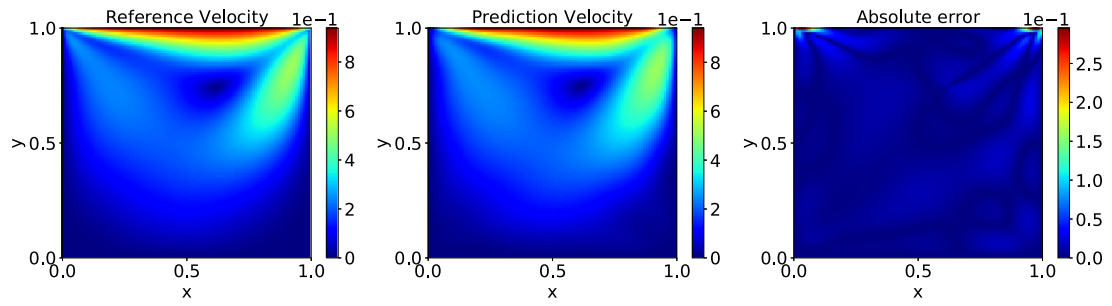


Fig. 9. Kovaszny flow: use Power function to solve the predictive solutions, exact solutions, and error distributions of Eq. (4.5).

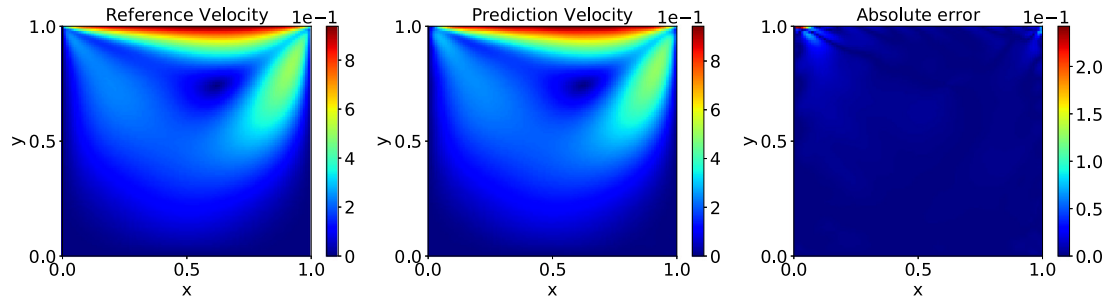
Table 4

Results by three types of polynomial training and the relative L^2 errors.

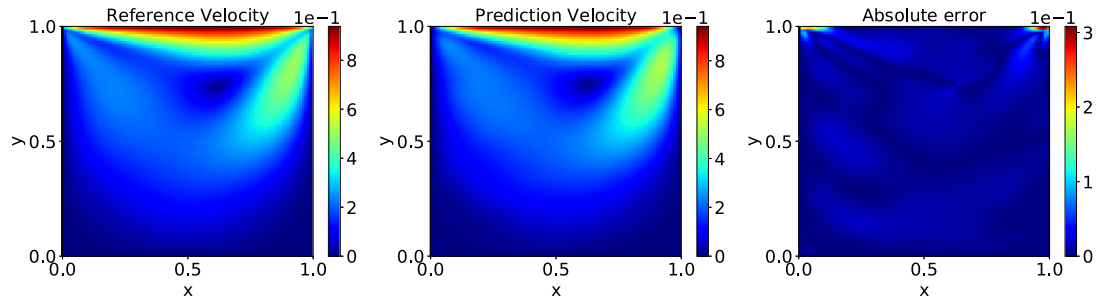
	Identified(clear data)	Identified(1% noise)	Identified(5% noise)
Exact		$(a, b) = (2, 4)$	
Power	(1.9969, 3.9906)	(1.9963, 3.9976)	(2.0206, 4.0521)
Legendre	(2.0000, 3.9981)	(1.9991, 4.0066)	(2.0219, 4.0597)
Chebyshev	(1.9994, 3.9963)	(1.9978, 4.0046)	(2.0260, 4.0559)
Power	$8.8542e-04$	$1.2690e-03$	$6.8992e-03$
Legendre	$1.4405e-04$	$1.1437e-03$	$7.0002e-03$
Chebyshev	$1.8808e-04$	$1.1631e-03$	$6.7828e-03$



(a) Power (Rel. L^2 error: 1.1045e-01)



(b) Legendre (Rel. L^2 error: 8.9456e-02)



(c) Chebyshev (Rel. L^2 error: 1.0593e-01)

Fig. 10. Flow in a lid-driven cavity: Reference solution, prediction solution, and absolute pointwise error.

Table 5
The relative L^2 errors based on three types of polynomial training.

	Identified(clear data)	Identified(1% noise)	Identified(5% noise)
Correct		$(a, b) = (5, -5)$	
Power	(4.9819, -4.9797)	(4.9795, -4.9772)	(5.0230, -5.0203)
Legendre	(4.9979, -4.9976)	(4.9928, -4.9923)	(5.0390, -5.0375)
Chebyshev	(4.9944, -4.9932)	(4.9895, -4.9886)	(5.0381, -5.0366)
Power	$1.1266e-03$	$6.1901e-04$	$3.3370e-03$
Legendre	$6.1901e-04$	$7.4403e-04$	$3.2767e-03$
Chebyshev	$6.1343e-04$	$7.4062e-04$	$3.2218e-03$

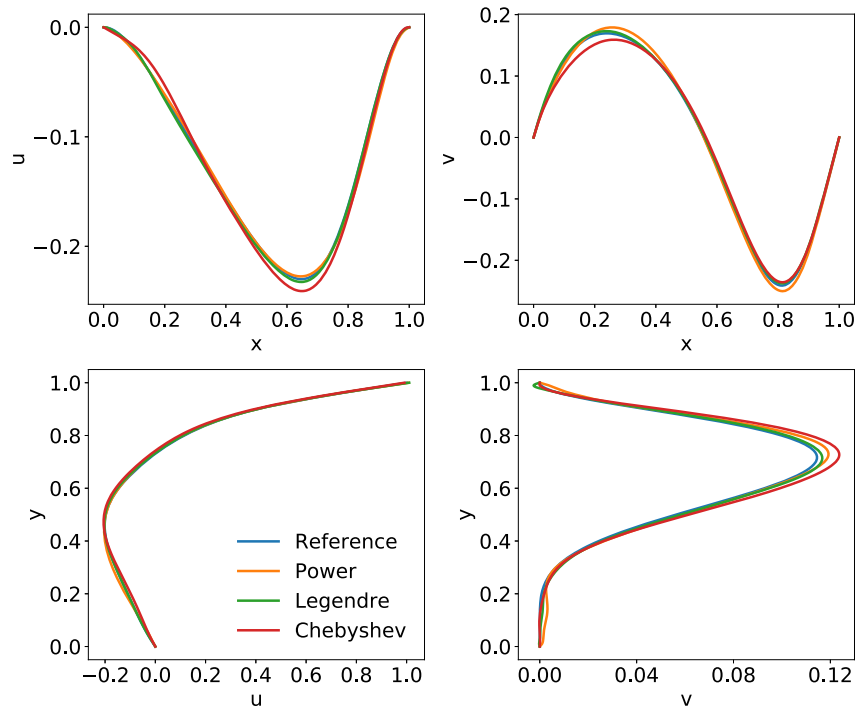


Fig. 11. Flow in a lid-driven cavity: Comparison of the centerline velocity obtained for the lid-driven cavity problem.

Table 6

Diffusion-reaction equation: use three types of polynomial training, resulting statistics for the learned parameter value of Eq. (4.9), and the L^2 errors.

	Identified(clear data)	Identified(1% noise)	Identified(5% noise)
Correct		$k = 300$	
Power	293.8383	303.0949	296.3561
Legendre	299.8051	299.5831	299.5352
Chebyshev	296.3506	296.0880	297.0784
Power	$1.2786e-02$	$1.1553e-02$	$9.6232e-03$
Legendre	$1.1494e-02$	$6.2961e-03$	$9.2300e-03$
Chebyshev	$1.7586e-02$	$8.9883e-03$	$1.2142e-02$

where u represents the concentration, $\lambda = 0.01$ is the diffusion coefficient and k is the reaction rate. The exact solution $u = \sin(\pi x)\sin(\pi y)\sin(\pi z)$ and f is obtained from the exact solution. We here assume that k is an unknown constant. The aim is to predict u in the entire computational domain and identify k . We use 5 neurons in each direction and 1000 training points. In Table 6, it shows the k values ‘discovered’ through neural network training, and L^2 errors on the test set. In Fig. 18, it shows the change of the parameter k with number of iterations. We can see that the PI-PINN gives very accurate results.

5. Conclusions

In this paper, we utilise the physics-informed neural networks (PINN) combined with interpolation polynomials to solve nonlinear partial differential equations. The structure of the neural network is simple and easy to be understood from the function space generated by the neural network based on polynomial basis functions to the polynomial function space. We consider use of orthogonal polynomials to construct the neural network. Several numerical experiments are investigated to check the efficiency of the neural network in solving nonlinear partial differential equations. To be specific, PI-PINN is used to simulate the Burgers equation, the Sine-Golden equation, the Allen-Cahn equation, the Kovasznay flow, and the Lid-driven cavity flow for forward problems. For the Burgers equation and the Allen-Cahn equation, the proposed approach improves accuracy. Similarly, for the Kovasznay

flow problem, PI-PINN gives a good result at $Re = 1000$. Compared with PINN, the accuracy is greatly improved. We then apply PI-PINN to solve inverse problems. Particularly, we investigate the performance of PI-PINN to discover the diffusion-reaction system at a reaction rate 300. The present method can estimate the unknown parameters and reconstruct the solution with high precision. Although the proposed method has achieved excellent results in several numerical examples, many problems remain to be explored, such as complex problems with large dimensions and problems with irregular geometric regions. In this paper, we only consider two kinds of relatively simple orthogonal polynomials to solve nonlinear partial differential equations on simple computational geometries for a more comprehensive validation purpose. In future, we will use PI-PINN to explore complex problems.

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Acknowledgements

This work is supported by the Natural Science Foundation of China (Grant Nos. 92152301, 12071406 and U19A2079) and the Research Fund from State Key Laboratory of High Temperature Gas Dynamics (2021KF02).

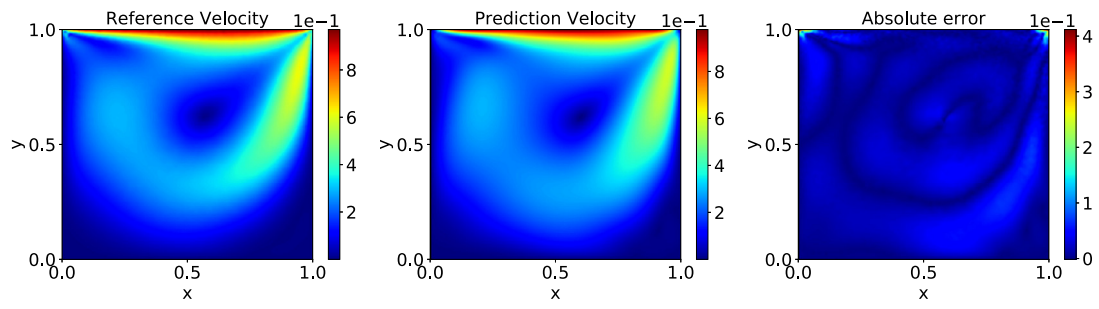
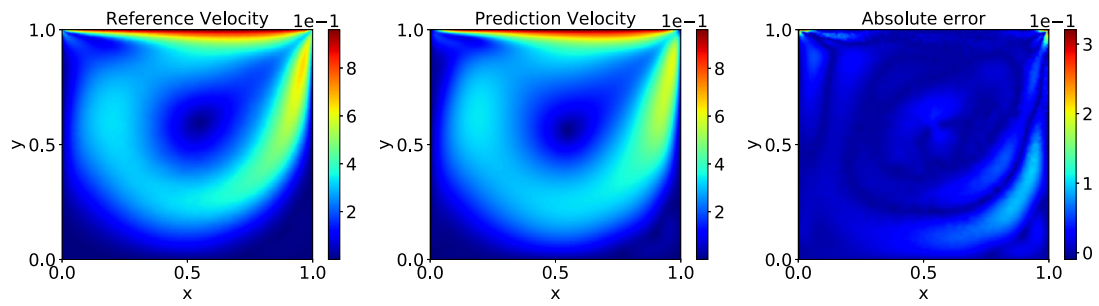
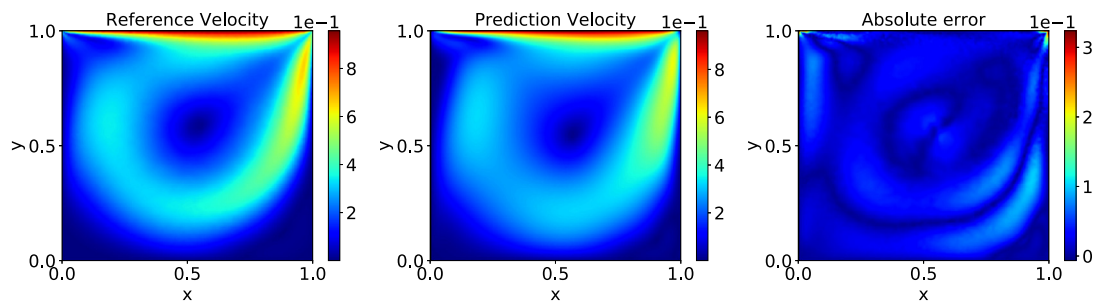
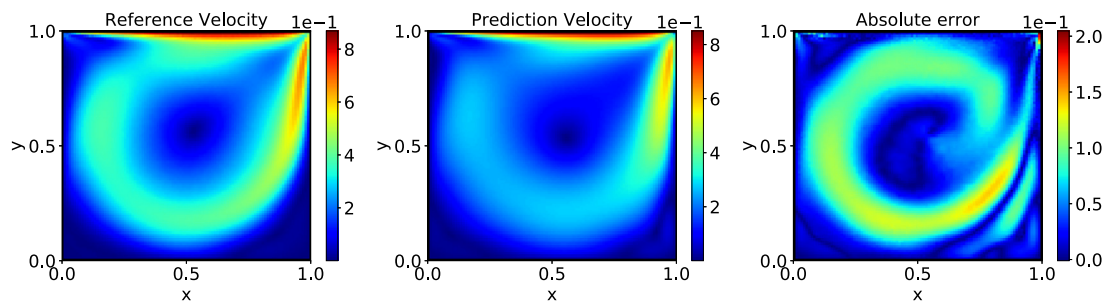
(a) $Re=300$ (Rel. L^2 error: $9.4115e-02$)(b) $Re=500$ (Rel. L^2 error: $9.7498e-02$)(c) $Re=600$ (Rel. L^2 error: $1.1458e-01$)(d) $Re=1000$ (Rel. L^2 error: $1.8394e-01$)

Fig. 12. Flow in a Lid-driven cavity: Reference solution, prediction solution and absolute pointwise error for different Reynolds numbers.

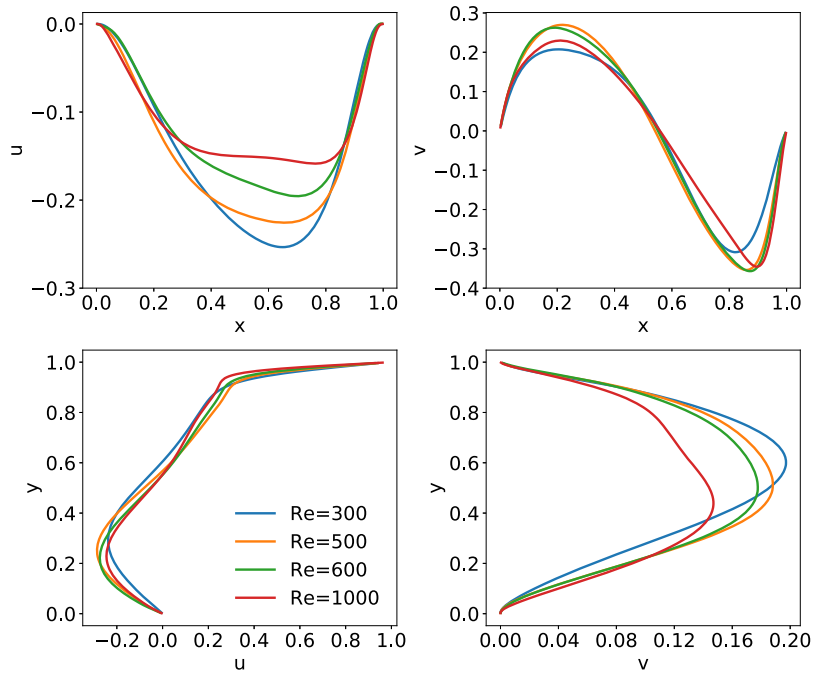


Fig. 13. Flow in a Lid-driven cavity: centerline velocity at different Reynolds numbers.

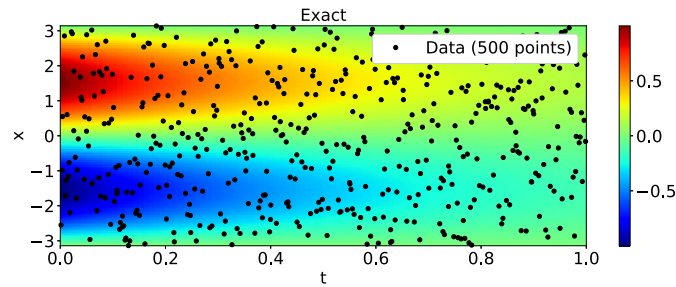


Fig. 14. Heat equation: the exact solution image of Eq. (4.7) and the training points.

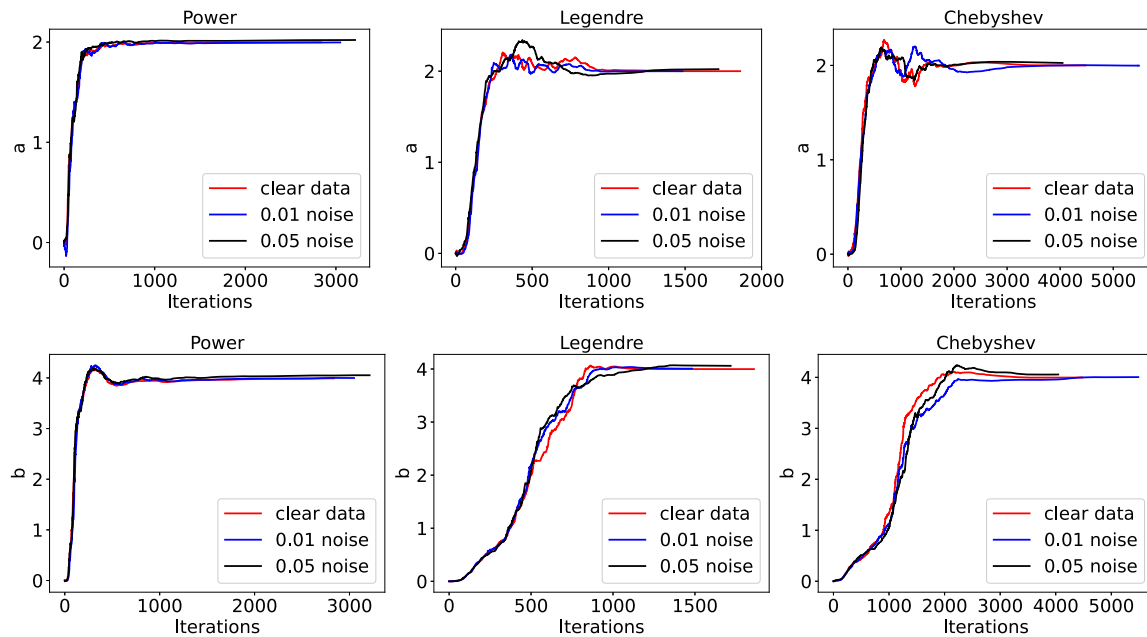


Fig. 15. Heat equation: the change curves of the parameters a and b .

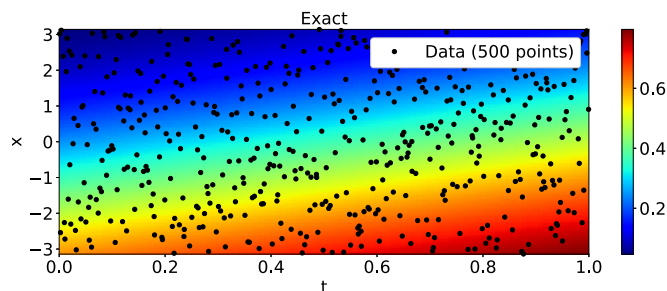


Fig. 16. KPP equation: the exact solution image of Eq. (4.8) and the training points.

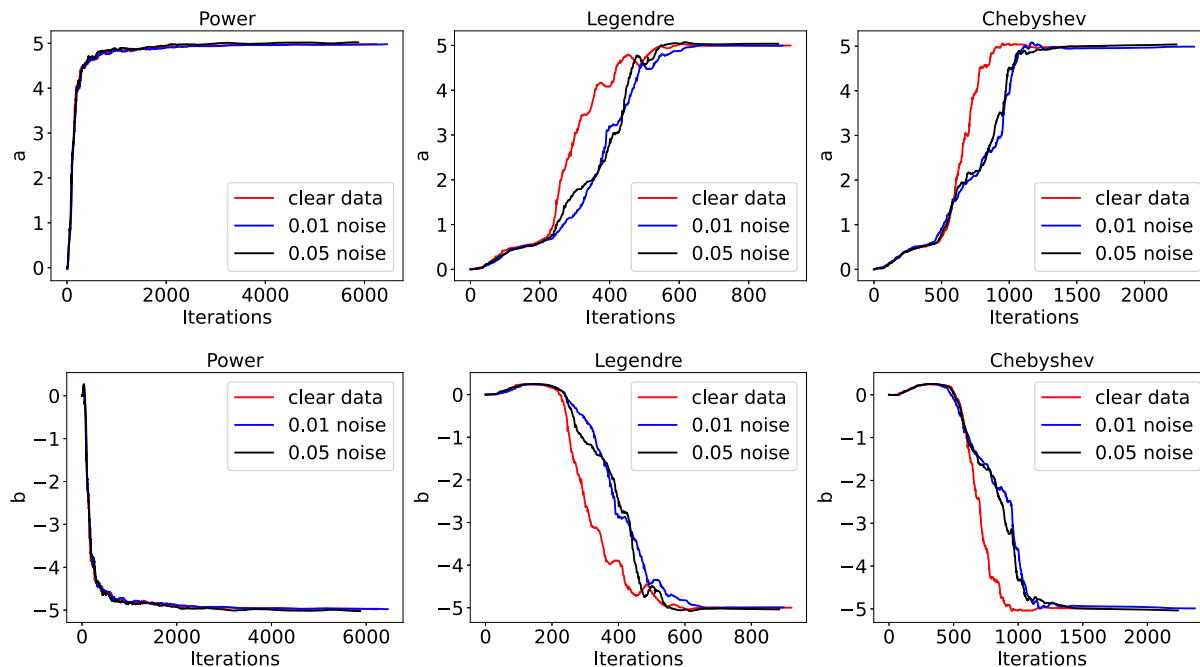


Fig. 17. KPP equation: the change curves of the parameters a and b .

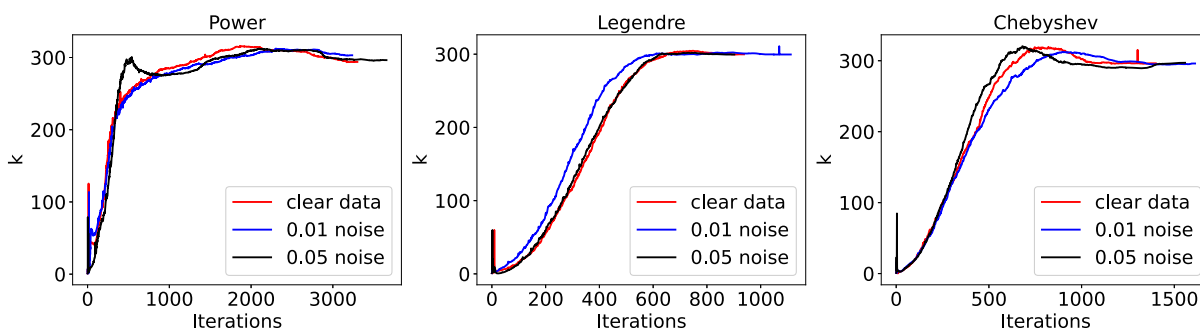


Fig. 18. Diffusion-reaction equation: the change curve of the parameter k .

References

- [1] S. Marsland, *Machine Learning*, CRC Press, Taylor & Francis Inc., Boca Raton, FL, 2014.
- [2] P. Mordohai, G.G. Medioni, Tensor voting: a perceptual organization approach to computer vision and machine learning, *Synthesis Lectures on Image Video & Multimedia Processing* 2 (1) (2006) 136.
- [3] P.H. Chen, H. Zafar, M.G. Aizenberg, T. Cook, Integrating natural language processing and machine learning algorithms to categorize oncologic response in radiology reports, *J. Digit. Imaging* 30 (2018) 178–184.
- [4] B. Alipanahi, A. Delong, M.T. Weirauch, B.J. Frey, Predicting the sequence specificities of dna- and rna-binding proteins by deep learning, *Nat. Biotechnol.* 33 (8) (2015) 831–839.
- [5] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2 (5) (1989) 359–366.
- [6] I.E. Lagaris, A. Likas, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Trans. Neural Netw.* 9 (5) (1998) 987–1000.
- [7] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.

- [8] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, Y. Liu, Physics-informed machine learning, *Nat. Rev. Phys.* 3 (June 2021) 422–440.
- [9] L. Yang, X. Meng, G.E. Karniadakis, B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data, *J. Comput. Phys.* 425 (2021) 109913.
- [10] X.H. Meng, H. Babaee, G.E. Karniadakis, Multi-fidelity bayesian neural networks: algorithms and applications, *J. Comput. Phys.* 438 (3) (2021) 110361.
- [11] A. Kovacs, et al., Conditional physics informed neural networks, *Commun. Nonlinear Sci. Numer. Simul.* 104 (2021) 106041.
- [12] E. Kharazmi, Z.Q. Zhang, G.E. Karniadakis, hp-vpinns: variational physics-informed neural networks with domain decomposition, *Comput. Methods Appl. Mech. Eng.* 374 (2021) 113547.
- [13] K. Li, K. Tang, T. Wu, Q. Liao, D3m: a deep domain decomposition method for partial differential equations, *IEEE Access* 8 (99) (2019) 5283–5294.
- [14] K. Shukla, A.D. Jagtap, G.E. Karniadakis, Parallel physics-informed neural networks via domain decomposition, *J. Comput. Phys.* 447 (2021) 110683.
- [15] A.D. Jagtap, E. Kharazmi, G.E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: applications to forward and inverse problems, *Comput. Methods Appl. Mech. Eng.* 365 (2020) 113028.
- [16] X. Meng, Z. Li, D. Zhang, G.E. Karniadakis, Ppinns: parareal physics-informed neural network for time-dependent pdes, *Comput. Methods Appl. Mech. Eng.* 370 (2020) 113250.
- [17] Z.P. Mao, A.D. Jagtap, G.E. Karniadakis, Physics-informed neural networks for high-speed flows, *Comput. Methods Appl. Mech. Eng.* 360 (2020) 112789.
- [18] S.F. Wang, H.W. Wang, P. Perdikaris, On the eigenvector bias of fourier feature networks: from regression to solving multi-scale pdes with physics-informed neural networks, *Comput. Methods Appl. Mech. Eng.* 384 (2021) 113938.
- [19] X.W. Jin, S.Z. Cai, H. Li, G.E. Karniadakis, Nsfnets (navier-stokes flow nets): physics-informed neural networks for the incompressible navier-stokes equations, *J. Comput. Phys.* 426 (2021) 109951.
- [20] Q. Lou, X.H. Meng, G.E. Karniadakis, Physics-informed neural networks for solving forward and inverse flow problems via the boltzmann-bgk formulation, *J. Comput. Phys.* 447 (2020) 110676.
- [21] D.E. Hurtado, F.S. Costabal, Y.B. Yang, P. Perdikaris, E. Kuhl, Physics-informed neural networks for cardiac activation mapping, *Front. Phys.* 8 (42) (2020).
- [22] Z. Fang, J. Zhan, Deep physical informed neural networks for metamaterial design, *IEEE Access* 8 (2019) 24506–24513.
- [23] R.G. Nascimento, K. Fricke, F. Viana, A tutorial on solving ordinary differential equations using python and hybrid physics-informed neural network, *Eng. Appl. Artif. Intell.* 96 (153) (2020) 103996.
- [24] Y.Y. Chen, L. Lu, G.E. Karniadakis, L.D. Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, *Opt. Express* 28 (8) (Apr 2020) 11618–11633.
- [25] K. Shukla, P. Leoni, J. Blackshire, D. Sparkman, G.E. Karniadakis, Physics-informed neural network for ultrasound nondestructive quantification of surface breaking cracks, *J. Nondestruct. Eval.* 39 (3) (2020) 1–20.
- [26] S.F. Wang, P. Perdikaris, Deep learning of free boundary and stefan problems, *J. Comput. Phys.* 428 (2021) 109914.
- [27] A.D. Jagtap, K.J. Kawaguchi, G.E. Karniadakis, Adaptive activation functions accelerate convergence in deep and physics-informed neural networks, *J. Comput. Phys.* 404 (2019) 109136.
- [28] A.D. Jagtap, K.J. Kawaguchi, G.E. Karniadakis, Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks, *Proc. R. Soc. A* 476 (2239) (2020) 20200334.
- [29] A.D. Jagtap, Y.J. Shin, K.J. Kawaguchi, G.E. Karniadakis, Deep kronecker neural networks: a general framework for neural networks with adaptive activation functions, *Neurocomputing* 468 (2022) 165–180.
- [30] S.F. Wang, Y.J. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, *SIAM J. Sci. Comput.* 43 (5) (2021) A3055–A3081.
- [31] S.F. Wang, X.L. Yu, P. Perdikaris, When and why pinns fail to train: a neural tangent kernel perspective, *J. Comput. Phys.* 449 (2022) 110768.
- [32] Z.X. Xiang, W. Peng, X. Liu, W. Yao, Self-adaptive loss balanced physics-informed neural networks, *Neurocomputing* 496 (2022) 11–34.
- [33] H.S. Tan, Fourier neural networks and generalized single hidden layer networks in aircraft engine fault diagnostics, *J. Eng. Gas Turbines Power* 128 (4) (2006) 773–782.
- [34] Z. Wei, Y. Zhu, L. Cai, Fourier-neural-network-based learning control for a class of nonlinear systems with flexible components, *IEEE Trans. Neural Netw.* 20 (1) (2009) 139–151.
- [35] Y.L. Yang, M.Z. Hou, et al., Neural network algorithm based on legendre improved extreme learning machine for solving elliptic partial differential equations, *Soft Comput.* 24 (2) (Jan 2020) 1083–1096.
- [36] S. Mall, S. Chakraverty, Single layer chebyshev neural network model for solving elliptic partial differential equations, *Neural Process. Lett.* 45 (3) (2017) 1–16.
- [37] A.G. Baydin, B.A. Pearlmutter, et al., Automatic differentiation in machine learning: a survey, *J. Mach. Learn. Res.* 18 (153) (2018) 1–43.
- [38] P.H. Chiu, J.C. Wong, C.C. Ooi, M.H. Dao, Y.S. Ong, Can-pinn: a fast physics-informed neural network based on coupled-automatic-numerical differentiation method, *Comput. Methods Appl. Mech. Eng.* 395 (2022) 114909.
- [39] L. Lu, X.H. Meng, Z.P. Mao, G.E. Karniadakis, Deepxde: a deep learning library for solving differential equations, *SIAM Rev.* 63 (1) (2021) 208–228.
- [40] E. Schiassi, R. Furfaro, C. Leake, M.D. Florio, D. Mortari, Extreme theory of functional connections: a fast physics-informed neural network method for solving ordinary and partial differential equations, *Neurocomputing* 457 (48) (2021).
- [41] L. Bottou, *Large-Scale Machine Learning with Stochastic Gradient Descent*, 2010, pp. 177–186.
- [42] X. Zeng, X.U. Xiang, H. Zhang, Estimation of coniferous forest biomass by bp neural network model based on small batch gradient descent, in: *Forest Inventory and Planning*, 2017.
- [43] D. Kingma, J. Ba, Adam: a method for stochastic optimization, *Comput. Sci.* (2014).
- [44] R.H. Byrd, P. Lu, J. Nocedal, C. Zhu, A limited memory algorithm for bound constrained optimization, *SIAM J. Sci. Comput.* 16 (5) (1995) 1190–1208.
- [45] R.P. Agarwal, D. O'Regan, *Legendre Polynomials and Functions*, Springer, New York, 2009, pp. 47–56.
- [46] T.J. Rivlin, The chebyshev polynomials, *Math. Comput.* 30 (134) (1976) 374.
- [47] Z.X. Xiang, W. Peng, X.H. Zheng, X.Y. Zhao, W. Yao, Self-adaptive loss balanced physics-informed neural networks for the incompressible navier-stokes equations, *Acta Mech. Sin.* 37 (1) (2021) 47–52.
- [48] X. Yang, Y. Ge, L. Zhang, A class of high-order compact difference schemes for solving the burgers' equations, *Appl. Math. Comput.* 358 (2019) 394–417.
- [49] M. Stein, Large sample properties of simulations using latin hypercube sampling, *Technometrics* 29 (2) (1987) 143–151.
- [50] N. Martinov, N. Vitanov, On some solutions of the two-dimensional sine-gordon equation, *J. Phys. A, Gen. Phys.* 25 (8) (1999) L419.
- [51] S.M. Allen, J.W. Cahn, A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening, *Acta Metall.* 27 (6) (1979) 1085–1095.
- [52] L. Golubovi, A. Levandovsky, D. Moldovan, Interface dynamics and far-from-equilibrium phase transitions in multilayer epitaxial growth and erosion on crystal surfaces: continuum theory insights, *East Asian J. Appl. Math.* 1 (04) (2011) 297–371.
- [53] R. Kobayashi, Modeling and numerical simulations of dendritic crystal growth, *Physica D* 63 (3–4) (1993) 410–423.
- [54] M. Sabir, A. Shah, W. Muhammad, I. Ali, P. Bastian, A mathematical model of tumor hypoxia targeting in cancer treatment and its numerical simulation, *Comput. Math. Appl.* 74 (12) (2017) 3250–3259.
- [55] L.G. Rebholz, S.A. Watro, A note on taylor-eddy and kovaszny solutions of ns-deconvolution and lera-deconvolution models, *J. Nonlinear Dyn.* 2014 (1) (2014) 1–5.
- [56] U. Ghia, K.N. Ghia, C.T. Shin, High-re solutions for incompressible flow using the navier-stokes equations and a multigrid method, *J. Comput. Phys.* 48 (3) (1982) 387–411.
- [57] J.R. Cannon, *The One-Dimensional Heat Equation*, Addison-Wesley Pub. Co, 12 1984.
- [58] M.L. Wang, H.X. Li, X. Chen, Y. Chen, Deep learning-based model reduction for distributed parameter systems, *IEEE Trans. Syst. Man Cybern. Syst.* 46 (12) (2016) 1664–1674.